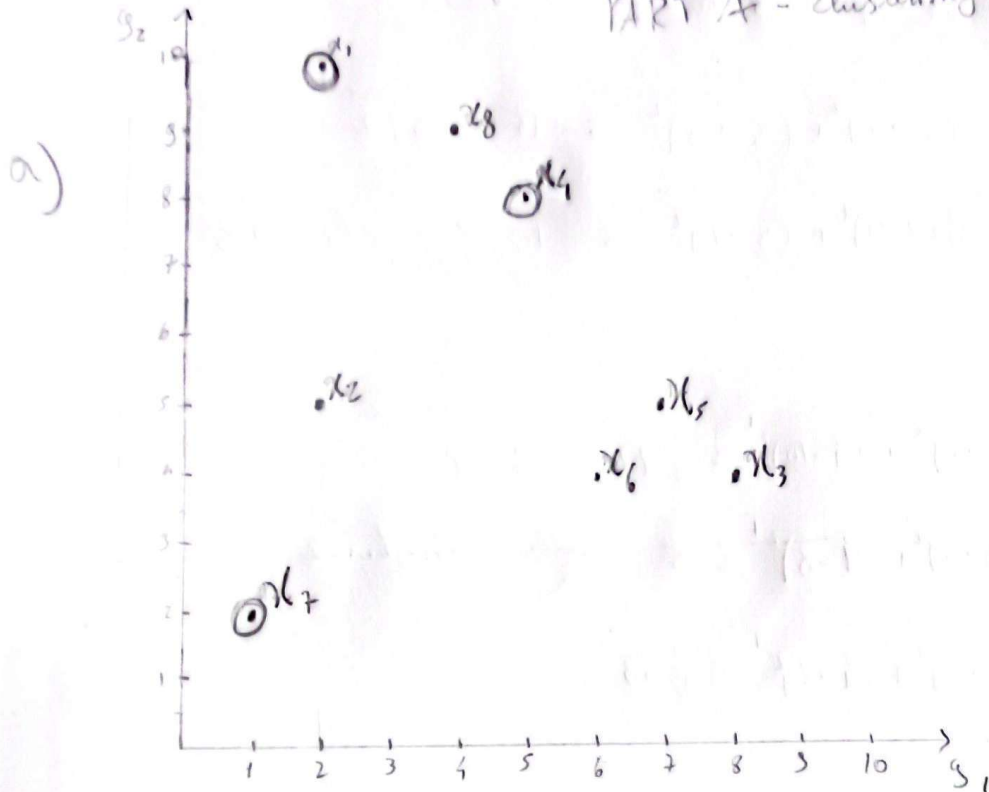


I Pen - and - Paper H W 4 G 48
PART A - clustering



b) $c_1 = (2, 10)$

$c_2 = (5, 8)$

$c_3 = (1, 2)$

d_1 :

$$d(x_1, c_1) = \sqrt{(2-2)^2 + (10-10)^2} = 0 \leftarrow \text{cluster 1}$$

$$d(x_1, c_2) = \sqrt{(2-5)^2 + (10-8)^2} = \sqrt{13}$$

$$d(x_1, c_3) = \sqrt{(2-1)^2 + (10-2)^2} = \sqrt{65}$$

x_2 :

$$d(x_2, c_1) = \sqrt{(2-2)^2 + (5-10)^2} = 5$$

$$d(x_2, c_2) = \sqrt{(2-5)^2 + (5-8)^2} = \sqrt{18} = 3\sqrt{2}$$

$$d(x_2, c_3) = \sqrt{(2-1)^2 + (5-2)^2} = \sqrt{10} \leftarrow \text{cluster 3}$$

x_3 :

$$d(x_3, c_1) = \sqrt{(8-2)^2 + (4-10)^2} = \sqrt{72} = 6\sqrt{2}$$

$$d(x_3, c_2) = \sqrt{(8-5)^2 + (4-8)^2} = 5 \leftarrow \text{cluster 2}$$

$$d(x_3, c_3) = \sqrt{(8-1)^2 + (4-2)^2} = \sqrt{53}$$

x_4 :

$$d(x_4, c_1) = \sqrt{(5-2)^2 + (8-10)^2} = \sqrt{13}$$

$$d(x_4, c_2) = \sqrt{(5-5)^2 + (8-8)^2} = 0 \leftarrow \text{cluster 2}$$

$$d(x_4, c_3) = \sqrt{(5-1)^2 + (8-2)^2} = \sqrt{52} = 2\sqrt{13}$$

x_5 :

$$d(x_5, c_1) = \sqrt{(7-2)^2 + (5-10)^2} = \sqrt{50} = 5\sqrt{2}$$

$$d(x_5, c_2) = \sqrt{(7-5)^2 + (5-8)^2} = \sqrt{13} \leftarrow \text{cluster 2}$$

$$d(x_5, c_3) = \sqrt{(7-1)^2 + (5-2)^2} = \sqrt{45} = 3\sqrt{5}$$

x_6 :

$$d(x_6, c_1) = \sqrt{(6-2)^2 + (4-10)^2} = \sqrt{52} = 2\sqrt{13}$$

$$d(x_6, c_2) = \sqrt{(6-5)^2 + (4-8)^2} = \sqrt{17} \leftarrow \text{cluster 2}$$

$$d(x_6, c_3) = \sqrt{(6-1)^2 + (4-2)^2} = \sqrt{29}$$

x_7 :

$$d(x_7, c_1) = \sqrt{(1-2)^2 + (2-10)^2} = \sqrt{65}$$

$$d(x_7, c_2) = \sqrt{(1-5)^2 + (2-8)^2} = \sqrt{52} = 2\sqrt{13}$$

$$d(x_7, c_3) = \sqrt{(1-1)^2 + (2-2)^2} = 0 \leftarrow \text{cluster 3}$$

x_8 :

$$d(x_8, c_1) = \sqrt{(4-2)^2 + (9-10)^2} = \sqrt{5}$$

$$d(x_8, c_2) = \sqrt{(4-5)^2 + (9-8)^2} = \sqrt{2} \leftarrow \text{cluster 2}$$

$$d(x_8, c_3) = \sqrt{(4-1)^2 + (9-2)^2} = \sqrt{58}$$

cluster 1: $\{x_1\}$

cluster 2: $\{x_3, x_4, x_5, x_6, x_8\}$

cluster 3: $\{x_2, x_7\}$

Recalcular os centróides

$$C_1 = \text{média de } \{x_1\} = (2, 10)$$

$$C_2 = \text{média de } \{x_3, x_4, x_5, x_6, x_8\}$$

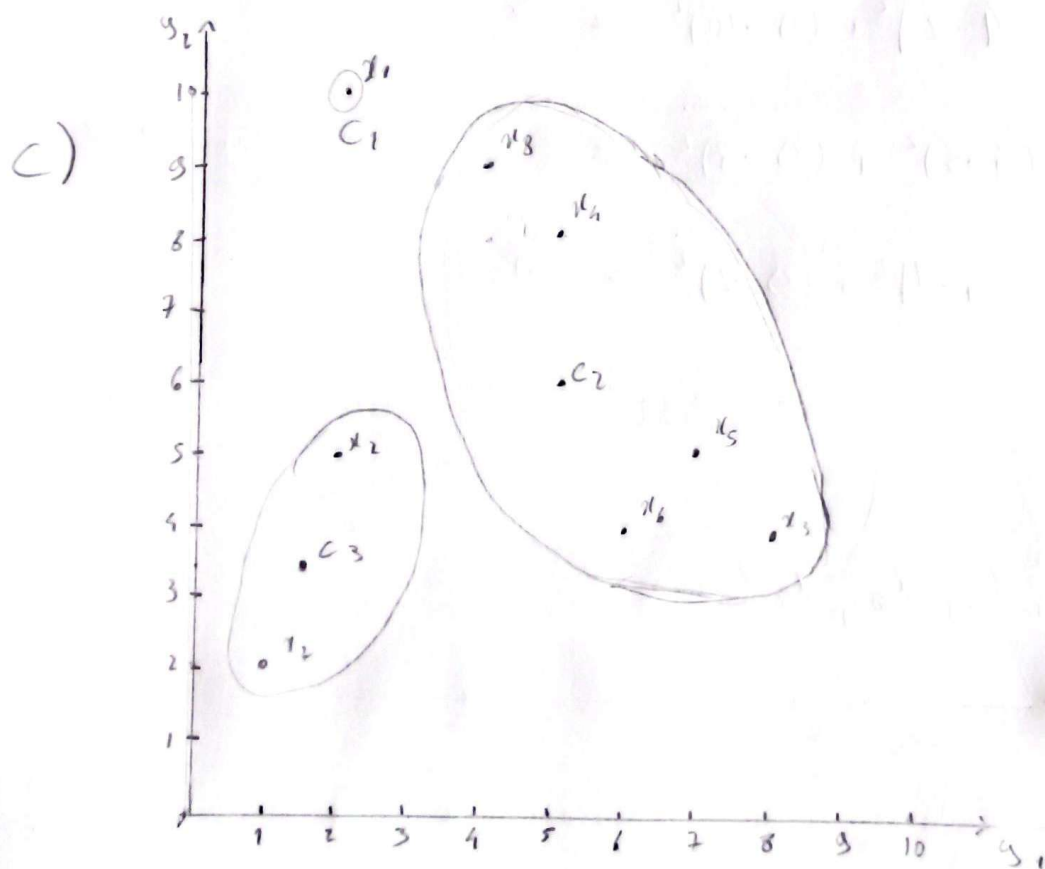
$$= ((8+5+7+6+4)/5, (4+8+5+4+9)/5)$$

$$C_2 = (6, 6)$$

$$C_3 = \text{média de } \{x_2, x_7\}$$

$$= ((2+1)/2, (5+2)/2)$$

$$C_3 = (1,5, 3,5)$$



d) A inicialização dos centróides tem impacto significativo no K-means nos seguintes pontos:

- Convergência para mínimos locais:
 - O K-means é sensível à inicialização e pode convergir para diferentes soluções dependendo dos centros iniciais. Não garante o ótimo global.
- Número de iterações:
 - Boas inicializações (próximas da solução final) levam a convergência mais rápida, no entanto más inicializações podem requerer muitas iterações.
- Qualidade da solução:
 - Centros iniciais muito próximos podem resultar em clusters desequilibrados ou vazios. Centros bem distribuídos tendem a gerar melhores partições.

PART B - PCA

a)

obs.	x_1	x_2	x_3	$\tilde{x}_1$	$\tilde{x}_2$	$\tilde{x}_3$
x_1	5	0	1	3	$-\frac{5}{3}$	1
x_2	0	5	0	-2	$\frac{10}{3}$	0
x_3	1	0	-1	-1	$-\frac{5}{3}$	-1
média	2	$\frac{5}{3}$	0	0	0	0

$$\tilde{X} = \begin{bmatrix} 3 & -\frac{5}{3} & 1 \\ -2 & \frac{10}{3} & 0 \\ -1 & -\frac{5}{3} & -1 \end{bmatrix}$$

$$\mu = \begin{bmatrix} 2 \\ \frac{5}{3} \\ 0 \end{bmatrix}$$

$$C = \frac{1}{N} \tilde{X}^T \tilde{X}$$

$$\tilde{X}^T = \begin{bmatrix} 3 & -2 & -1 \\ -\frac{5}{3} & \frac{10}{3} & -\frac{5}{3} \\ 1 & 0 & -1 \end{bmatrix}$$

$$\tilde{X}^T \tilde{X} = \begin{bmatrix} 3 & -2 & -1 \\ -\frac{5}{3} & \frac{10}{3} & -\frac{5}{3} \\ 1 & 0 & -1 \end{bmatrix} \begin{bmatrix} 3 & -\frac{5}{3} & 1 \\ -2 & \frac{10}{3} & 0 \\ -1 & -\frac{5}{3} & -1 \end{bmatrix}$$

$$= \begin{bmatrix} 14 & -10 & 4 \\ -10 & \frac{50}{3} & 0 \\ 4 & 0 & 2 \end{bmatrix}$$

$$C = \frac{1}{N} \tilde{X}^T \tilde{X} = \begin{bmatrix} \frac{14}{3} & -\frac{10}{3} & \frac{4}{3} \\ -\frac{10}{3} & \frac{50}{9} & 0 \\ \frac{4}{3} & 0 & \frac{2}{3} \end{bmatrix}$$

b) calcul des valeurs propres :

$$\det(C - \lambda I) = 0$$

$$\det \left(\begin{bmatrix} \frac{14}{3} - \lambda & -\frac{10}{3} & \frac{4}{3} \\ -\frac{10}{3} & \frac{50}{9} - \lambda & 0 \\ \frac{4}{3} & 0 & \frac{2}{3} - \lambda \end{bmatrix} \right) = 0$$

Expandindo pela 3ª linha

$$= \frac{4}{3} \det \begin{pmatrix} -\frac{10}{3} & \frac{50}{3} - \lambda \\ \frac{4}{3} & 0 \end{pmatrix} - 0 + \left(\frac{2}{3} - \lambda\right) \times \det \begin{pmatrix} \frac{14}{3} - \lambda & -\frac{10}{3} \\ -\frac{14}{3} & \frac{50}{3} - \lambda \end{pmatrix}$$

$$= -\frac{10}{3} \times 0 - \left(\frac{50}{3} - \lambda\right) \times \frac{4}{3} =$$

$$= -\frac{200}{27} + \frac{4}{3}\lambda$$

$$\left(\frac{14}{3} - \lambda\right)\left(\frac{50}{3} - \lambda\right) - \frac{100}{9}$$

$$= \frac{700}{27} - \frac{50}{9}\lambda - \frac{14}{3}\lambda + \lambda^2 - \frac{100}{9}$$

$$= \frac{400}{27} - \frac{92}{9}\lambda + \lambda^2$$

Substitua tudo:

$$\frac{4}{3} \left(-\frac{200}{27} + \frac{4}{3}\lambda\right) + \left(\frac{2}{3} - \lambda\right) \left(\frac{400}{27} - \frac{92}{9}\lambda + \lambda^2\right) = 0$$

$$\Rightarrow -\frac{800}{81} + \frac{16}{9}\lambda + \frac{800}{81} - \frac{184}{27}\lambda + \frac{2}{3}\lambda^2 - \frac{400}{27}\lambda + \frac{92}{9}\lambda^2 - \lambda^3 = 0$$

$$\Rightarrow -\frac{536}{27}\lambda + \frac{98}{9}\lambda^2 - \lambda^3 = 0$$

$$\lambda_1 = 8,54$$

$$\lambda_2 = 2,32$$

$$\lambda_3 = 0$$

Cálculo dos vetores próprios

Para $\lambda_1 = 8,57$

$$(\mathbb{C} - 8,57 \mathbb{I}) \vec{v}_1 = 0$$

$$\begin{bmatrix} \frac{14}{3} - 8,57 & -\frac{10}{3} & \frac{4}{3} \\ -\frac{10}{3} & \frac{50}{3} - 8,57 & 0 \\ \frac{4}{3} & 0 & \frac{2}{3} - 8,57 \end{bmatrix} \begin{bmatrix} v_{11} \\ v_{12} \\ v_{13} \end{bmatrix} = 0$$

$$\frac{4}{3} v_{11} - \left(\frac{2}{3} - 8,57\right) v_{13} = 0 \Leftrightarrow \frac{4}{3} v_{11} = -7,90 v_{13} \Leftrightarrow v_{11} = 5,93 v_{13}$$

$$-3,90 v_{11} - \frac{10}{3} v_{12} + \frac{4}{3} v_{13} = 0 \Leftrightarrow$$

$$\Leftrightarrow -3,90(5,93 v_{13}) - \frac{10}{3} v_{12} + \frac{4}{3} v_{13} = 0$$

$$\Leftrightarrow -23,13 v_{13} + \frac{4}{3} v_{13} = \frac{10}{3} v_{12} \Leftrightarrow$$

$$\Leftrightarrow v_{12} = -6,54 v_{13}$$

Escolhendo $v_{13} = 1$

$$\vec{v}_1 = (5,93, -6,54, 1)$$

$$\|\vec{v}_1\| = \sqrt{(5,93)^2 + (-6,54)^2 + 1^2} = \sqrt{78,9} \simeq 8,88$$

$$\vec{v}_1 = (0,667; -0,736; 0,113)$$

Para $\lambda_2 = 2,32$

$$\begin{bmatrix} \frac{14}{3} - 2,32 & -\frac{10}{3} & \frac{4}{3} \\ -\frac{10}{3} & \frac{50}{3} - 2,32 & 0 \\ \frac{4}{3} & 0 & \frac{2}{3} - 2,32 \end{bmatrix} \begin{bmatrix} v_{21} \\ v_{22} \\ v_{23} \end{bmatrix} = 0$$

$$\frac{4}{3} v_{21} - \left(\frac{2}{3} - 2,32\right) v_{23} = 0 \Leftrightarrow v_{21} = 1,24 v_{23}$$

$$-\frac{10}{3} v_{21} + \left(\frac{50}{3} - 2,32\right) v_{22} = 0 \Leftrightarrow v_{22} = 1,277 v_{23}$$

Escolhendo $v_{23} = 1$

$$v_2 = (1,24; 1,277; 1)$$

$$\|v_2\| = \sqrt{1,24^2 + 1,277^2 + 1^2} \approx 2,042$$

$$v_2 = (0,607; 0,625; 0,490)$$

$$W = \begin{bmatrix} 0,607 & 0,607 \\ -0,736 & 0,625 \\ 0,113 & 0,490 \end{bmatrix}$$

Este plano minimiza o erro de projeção porque, captura a máxima variância dos dados ($\lambda_1 + \lambda_2 = 10,83$ de um total de $10,83$) e como $\lambda_3 = 0$ significa que não há variância perdida na 3ª dimensão (os dados são essencialmente 2D)

$$c) \quad \tilde{x} = \begin{bmatrix} 3 & -\frac{5}{3} & 1 \\ -2 & \frac{10}{3} & 0 \\ -1 & -\frac{5}{3} & -1 \end{bmatrix}$$

Projeção no plano

Multiplicar cada ponto com os pesos dados propostos

Para $\tilde{x}_1$

$$PC1 = 3 \times 0,667 + (-\frac{5}{3}) \times (-0,736) + 1 \times 0,113 \approx 3,34$$

$$PC2 = 3 \times 0,607 + (-\frac{5}{3}) \times 0,625 + 1 \times 0,490 \approx 1,27$$

Para $\tilde{x}_2$

$$PC1 = -2 \times 0,667 + \frac{10}{3} \times (-0,736) + 0 \approx -3,79$$

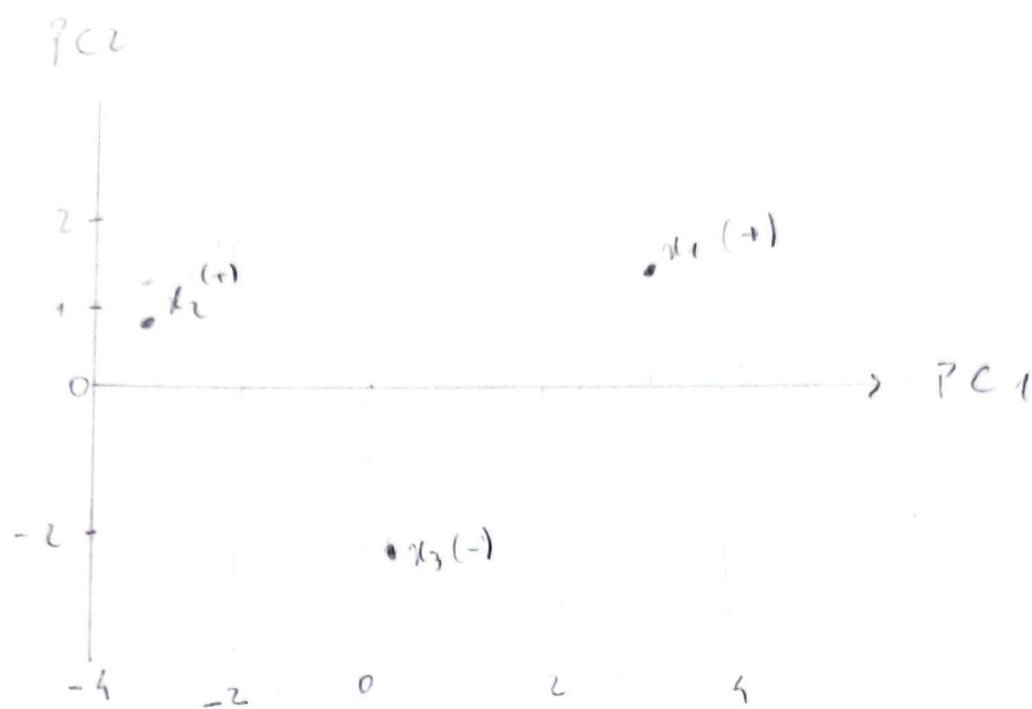
$$PC2 = -2 \times 0,607 + \frac{10}{3} \times 0,625 + 0 \approx 0,87$$

Para $\tilde{x}_3$

$$PC1 = -1 \times 0,667 + (-\frac{5}{3}) \times (-0,736) + (-1) \times 0,113 \approx 0,45$$

$$PC2 = -1 \times 0,607 + (-\frac{5}{3}) \times 0,625 + (-1) \times 0,490 \approx -2,14$$

Plot



O plano ajuda a discriminar entre as classes, nos eixos de PC2 (a 2ª componente principal).

Uma linha horizontal $PC2 = 0$ separa perfeitamente as duas classes, no entanto os pontos da classe + estão espacialmente distantes em PC1, partilhando valores positivos de PC2. O ponto da classe - tem valor negativo de PC2, separando-se claramente dos outros pontos.

II. Programming

Part A: Clustering:

1.

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, f1_score
from sklearn.preprocessing import MinMaxScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA

# Select all variables, but outcome
df = pd.read_csv("diabetes.csv")
X = df.drop(columns=['Outcome'])
Y = df['Outcome']

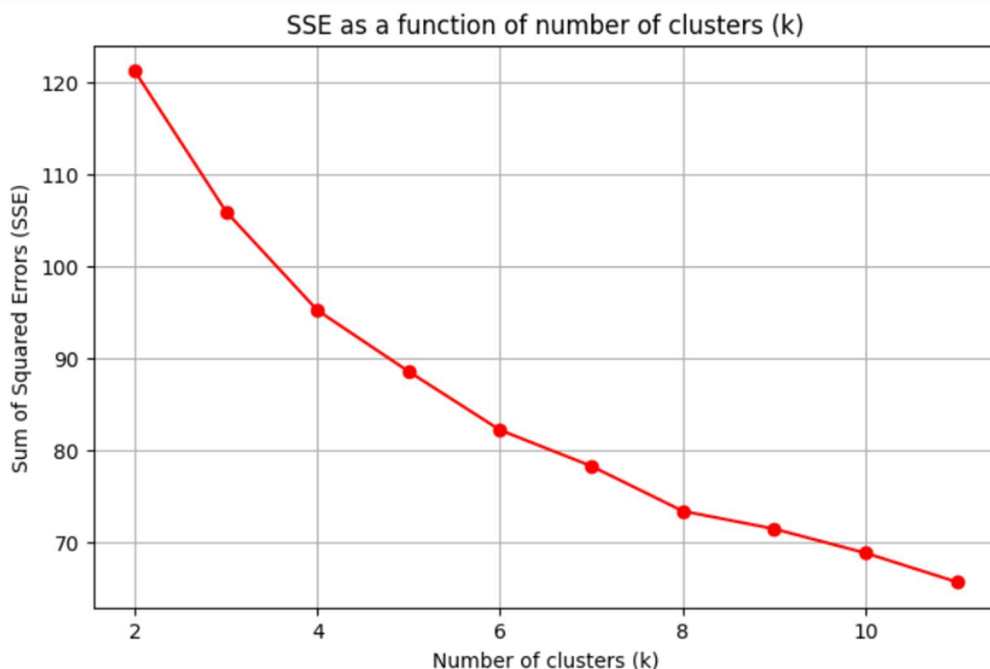
# Scale X variables with MinMax
scaler = MinMaxScaler()
X_normalized = scaler.fit_transform(X)
```

```
k_values = [2, 3, 4, 5, 6, 7, 8, 9, 10, 11]
sse_values = []

for k in k_values:
    kmeans = KMeans(n_clusters=k, max_iter = 500, random_state=42)
    kmeans.fit(X_normalized)

    sse_values.append(kmeans.inertia_)

# Plot SSE as a function of k
plt.figure(figsize=(8, 5))
plt.plot(k_values, sse_values, marker='o', linestyle='-', color='r')
plt.xlabel('Number of clusters (k)')
plt.ylabel('Sum of Squared Errors (SSE)')
plt.title('SSE as a function of number of clusters (k)')
plt.grid(True)
plt.show()
```



2.

```
kmeans = KMeans(n_clusters=6, random_state=42)
kmeans.fit(X_normalized) # model is trained for k=6

# Build a dataframe with outcome + cluster of each observation
normalized_df = pd.DataFrame(X_normalized, columns=X.columns, index=X.index)
normalized_df['Cluster'] = kmeans.labels_
normalized_df['Outcome'] = Y

# Take the most common outcome in each cluster
outcome_per_cluster = normalized_df.groupby('Cluster')['Outcome'].agg(lambda x: x.mode()[0])

# Assign the most common outcome in a cluster to all values in that cluster
clustered_df = normalized_df
clustered_df['Outcome'] = clustered_df['Cluster'].map(outcome_per_cluster)

Y_pred = clustered_df['Outcome']

conf_matrix = confusion_matrix(Y, Y_pred)
acc = accuracy_score(Y, Y_pred)
prec = precision_score(Y, Y_pred)
rec = recall_score(Y, Y_pred)
f1 = f1_score(Y, Y_pred)

print("Confusion Matrix:\n", conf_matrix)
print("Accuracy:", acc)
print("Precision:", prec)
print("Recall:", rec)
print("F1-score:", f1)
```

Confusion Matrix:

```
[[390 110]
```

```
[119 149]]
```

Accuracy: 0.7018229166666666

Precision: 0.5752895752895753

Recall: 0.5559701492537313

F1-score: 0.5654648956356736

Este não é um bom modelo de classificação. Embora a accuracy seja 70%, as métricas críticas são fracas:

- Precision (57.5%) e Recall (55.6%) são insuficientes - o modelo falha em identificar corretamente 44% dos casos de diabetes (119 falsos negativos);
- No contexto médico, isto é crítico: pacientes diabéticos não diagnosticados não recebem tratamento.

Limitações do clustering para classificação:

- Não usa labels no treino - agrupa apenas por similaridade das features;
- Clusters heterogêneos - podem conter ambas as classes, gerando erros ao aplicar a regra da maioria;
- Performance inferior a modelos supervisionados (que tipicamente atingem >75% accuracy neste dataset).

Conclusão: Clustering é útil para exploração de dados, mas inadequado para classificação, especialmente em aplicações críticas. Métodos supervisionados (Logistic Regression, Random Forest) são muito mais apropriados.

3.

```
# Train KMeans with k=6
kmeans = KMeans([n_clusters=6, random_state=42])
kmeans.fit(X_normalized)

# Create dataframe with clusters and original outcome
result_df = pd.DataFrame(X_normalized, columns=X.columns)
result_df['Cluster'] = kmeans.labels_
result_df['Outcome'] = Y.values

# 1. Class distribution per cluster
print("="*60)
print("CLASS DISTRIBUTION PER CLUSTER:")
print("="*60)
class_dist = result_df.groupby('Cluster')['Outcome'].value_counts().unstack(fill_value=0)
class_dist.columns = ['No Diabetes', 'Diabetes']
class_dist['Total'] = class_dist.sum(axis=1)
class_dist['% Diabetes'] = (class_dist['Diabetes'] / class_dist['Total'] * 100).round(2)
print(class_dist)
print()

# 2. Cluster centers
print("="*60)
print("CLUSTER CENTERS (Normalized):")
print("="*60)
centroids_df = pd.DataFrame(kmeans.cluster_centers_, columns=X.columns)
centroids_df.index.name = 'Cluster'
print(centroids_df.round(3))
print()
```

```
# 3. Identify most discriminative clusters
print("="*60)
print("MOST DISCRIMINATIVE CLUSTERS:")
print("="*60)
class_dist_sorted = class_dist.sort_values('% Diabetes', ascending=False)
print("Most discriminative = clusters with highest/lowest % Diabetes")
print(f"\nTop 3 by discrimination:")
print(f"- Cluster {class_dist_sorted.index[0]}: {class_dist_sorted['% Diabetes'].iloc[0]:.1f}% diabetes (high risk)")
print(f"- Cluster {class_dist_sorted.index[-1]}: {class_dist_sorted['% Diabetes'].iloc[-1]:.1f}% diabetes (low risk)")
print(f"- Cluster {class_dist_sorted.index[1]}: {class_dist_sorted['% Diabetes'].iloc[1]:.1f}% diabetes")
```

```
=====
CLASS DISTRIBUTION PER CLUSTER:
=====

      No Diabetes  Diabetes  Total  % Diabetes
Cluster
0              61       73    134    54.48
1             254       26    280     9.29
2              43       26     69    37.68
3              19       16     35    45.71
4              74       51    125    40.80
5              49       76    125    60.80

=====
CLUSTER CENTERS (Normalized):
=====

      Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin    BMI  \
Cluster
0         0.111    0.751         0.610         0.345    0.248  0.552
1         0.104    0.498         0.549         0.239    0.069  0.447
2         0.350    0.679         0.659         0.052    0.010  0.440
3         0.213    0.592         0.000         0.015    0.000  0.383
4         0.283    0.618         0.604         0.017    0.011  0.474
5         0.502    0.653         0.628         0.320    0.143  0.511
```

```
      DiabetesPedigreeFunction    Age
...
Top 3 by discrimination:
- Cluster 5: 60.8% diabetes (high risk)
- Cluster 1: 9.3% diabetes (low risk)
- Cluster 0: 54.5% diabetes
```

Clusters mais discriminativos:

- Cluster 5 (60.8% diabetes): Pacientes de alto risco com elevado número de Pregnancies (0.502), Glucose alta (0.653), maior idade (0.381) e BMI elevado (0.511). Perfil característico de diabetes associada a múltiplas gestações e fatores metabólicos acumulados.
- Cluster 1 (9.3% diabetes): População muito saudável com Glucose moderada-baixa (0.498), BMI baixo (0.447), poucos Pregnancies (0.104) e idade jovem (0.070). Insulina muito baixa (0.069) indica boa resposta metabólica.
- Cluster 0 (54.5% diabetes): Perfil de risco metabólico elevado com Glucose muito alta (0.751), BloodPressure elevada (0.610) e BMI alto (0.552). Apesar da idade baixa (0.135), apresenta fatores de risco críticos para diabetes tipo 2.

Conclusão: Os clusters separam claramente perfis de risco. Glucose, Pregnancies, BMI e Age são as features mais discriminativas, identificando população saudável jovem vs. perfis de alto risco metabólico.

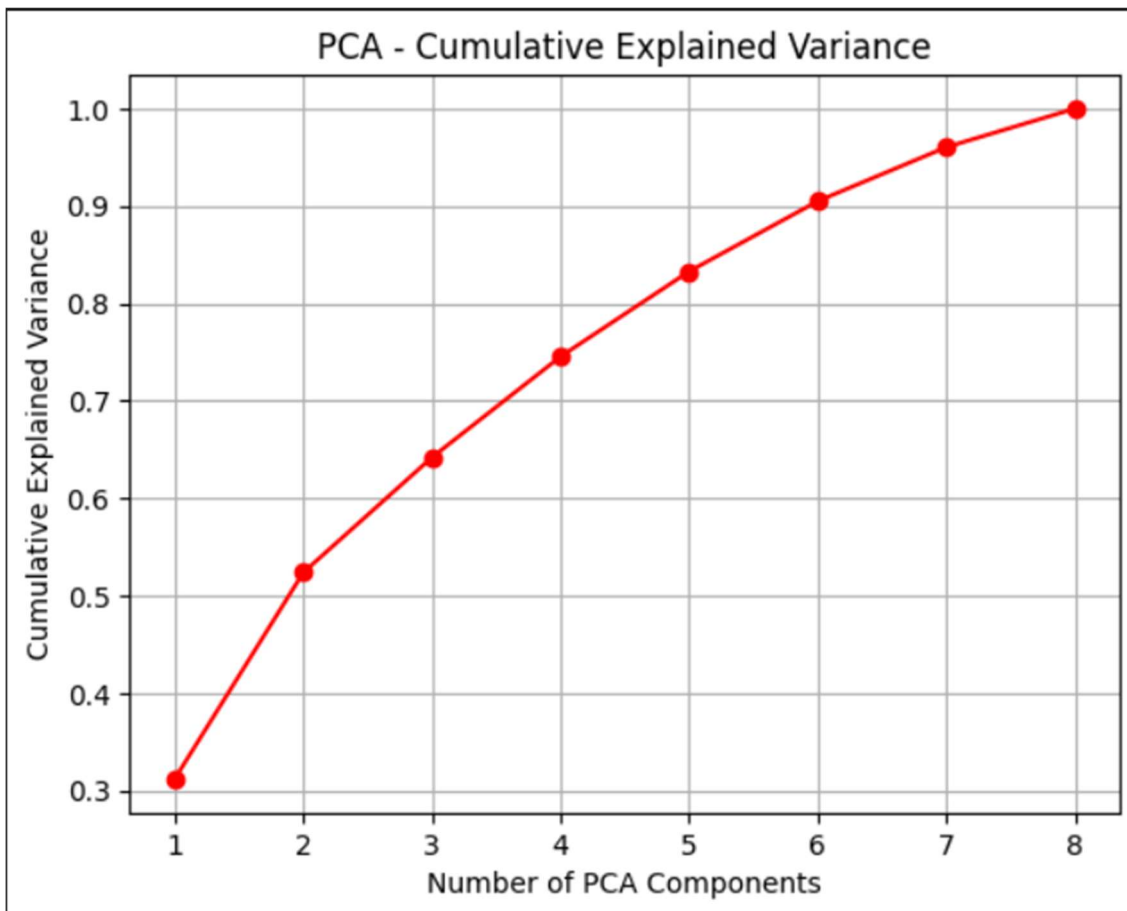
Part B: PCA:

1.

```
# Apply PCA to the normalized features
pca = PCA(random_state=42)
X_pca = pca.fit_transform(X_normalized)

# Compute cumulative explained variance
cumulative_variance = np.cumsum(pca.explained_variance_ratio_)

plt.figure()
plt.plot(range(1, len(cumulative_variance) + 1), cumulative_variance, marker='o', linestyle='-', color='r')
plt.xlabel('Number of PCA Components')
plt.ylabel('Cumulative Explained Variance')
plt.title('PCA - Cumulative Explained Variance')
plt.grid(True)
plt.show()
```



Para explicar pelo menos 80% da variância total, é necessário reter 5 componentes.

2.

```
# PCA with 1 component
pca = PCA(n_components=1, random_state=42)
pc1 = pca.fit_transform(X_normalized)

# Create dataframe
pc1_df = pd.DataFrame({
    'PC1': pc1.flatten(),
    'Outcome': Y.values
})

# Plot distribution
plt.figure(figsize=(10, 5))

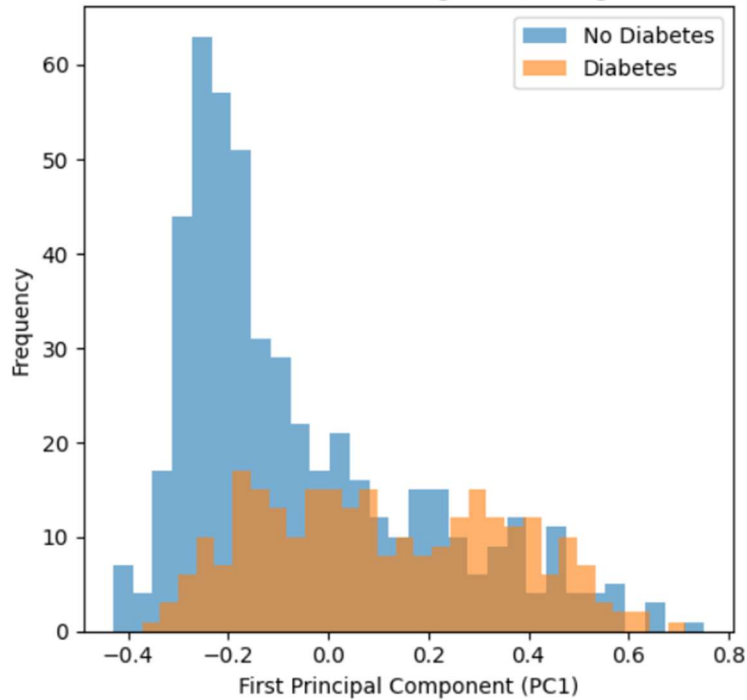
# Option 1: Histogram
plt.subplot(1, 2, 1)
plt.hist(pc1_df[pc1_df['Outcome'] == 0]['PC1'], alpha=0.6, label='No Diabetes', bins=30)
plt.hist(pc1_df[pc1_df['Outcome'] == 1]['PC1'], alpha=0.6, label='Diabetes', bins=30)
plt.xlabel('First Principal Component (PC1)')
plt.ylabel('Frequency')
plt.title('Class Distribution Along PC1 (Histogram)')
plt.legend()

# Option 2: KDE plot
plt.subplot(1, 2, 2)
sns.kdeplot(data=pc1_df[pc1_df['Outcome'] == 0]['PC1'], label='No Diabetes', fill=True)
sns.kdeplot(data=pc1_df[pc1_df['Outcome'] == 1]['PC1'], label='Diabetes', fill=True)
plt.xlabel('First Principal Component (PC1)')
plt.ylabel('Density')
plt.title('Class Distribution Along PC1 (KDE)')
plt.legend()

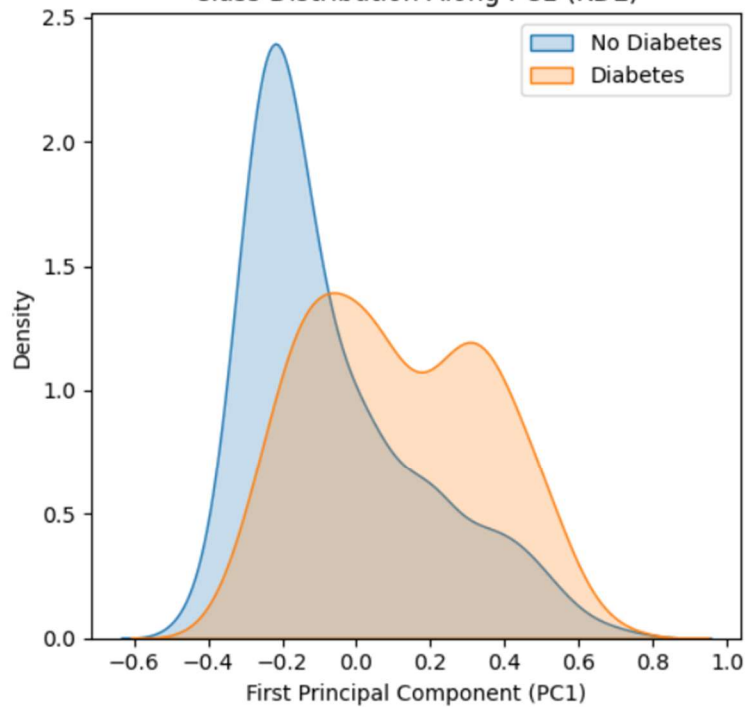
plt.tight_layout()
plt.show()

# Print explained variance
print(f"Variance explained by PC1: {pca.explained_variance_ratio_[0]:.4f} ({pca.explained_variance_ratio_[0]*100:.2f}%)")
```

Class Distribution Along PC1 (Histogram)



Class Distribution Along PC1 (KDE)



Variance explained by PC1: 0.3119 (31.19%)

PC1 explica apenas 31.19% da variância total e mostra sobreposição significativa entre as classes. A distribuição de pacientes diabéticos está mais dispersa e deslocada para valores positivos de PC1, enquanto não-diabéticos concentram-se em valores negativos. No entanto, a separação é fraca - há considerável overlap entre as classes.

Isto é esperado porque PCA é não-supervisionado: maximiza variância total dos dados, não a separabilidade entre classes. A primeira componente captura a direção de maior variação global, que não corresponde necessariamente à melhor discriminação entre diabéticos e não-diabéticos.

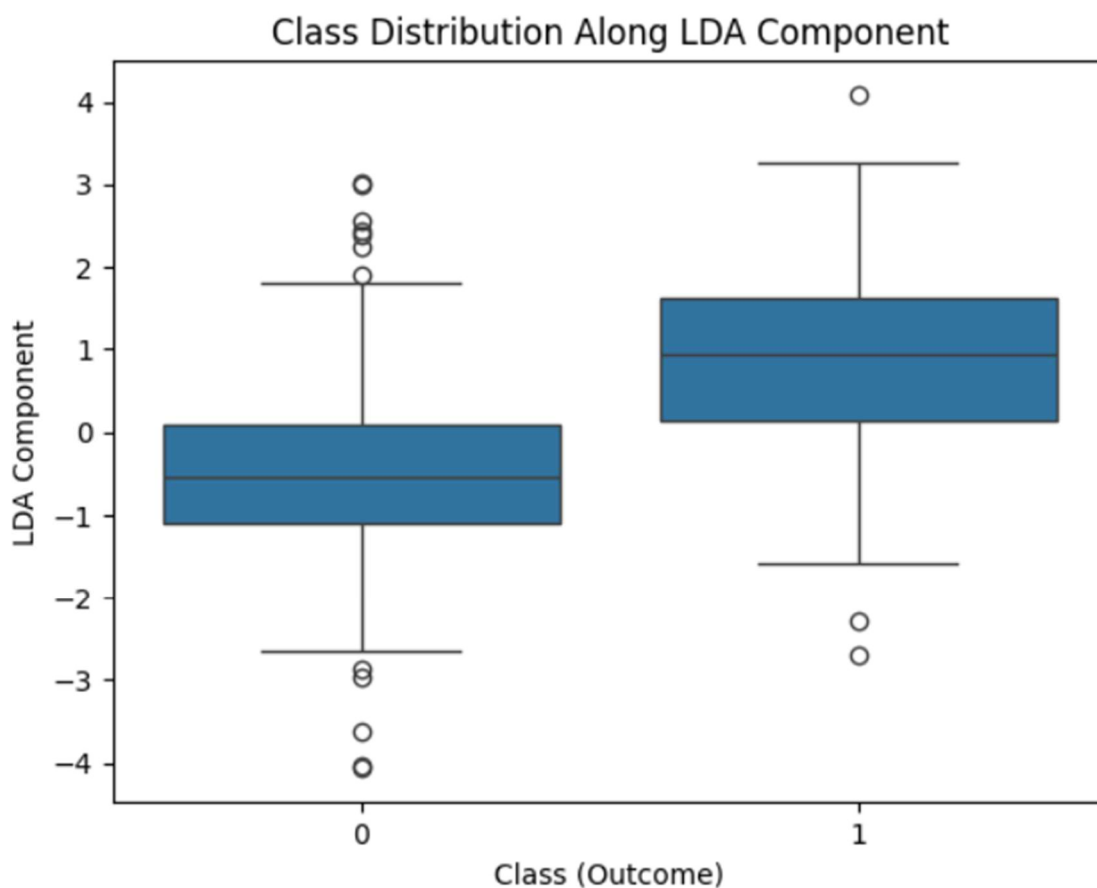
Conclusão: PC1 não é adequado para discriminar eficazmente as duas classes, evidenciando as limitações de métodos não-supervisionados para tarefas de classificação.

3.

```
lda = LDA(n_components=1) # for binary classification, only 1 component is possible
X_lda = lda.fit_transform(X_normalized, Y)

df_lda = pd.DataFrame({
    ... 'LDA1': X_lda.flatten(),
    ... 'Outcome': Y.values
})

plt.figure()
sns.boxplot(x='Outcome', y='LDA1', data=df_lda)
plt.title('Class Distribution Along LDA Component')
plt.xlabel('Class (Outcome)')
plt.ylabel('LDA Component')
plt.show()
```



O método LDA mostra separação clara entre as classes, com medianas bem distintas: não-diabéticos centrados em ~ -0.6 e diabéticos em ~ 1.0 . Embora exista alguma sobreposição (visível nos outliers e nas caudas das distribuições), a discriminação é significativamente melhor que PC1. Isto ocorre porque o LDA é supervisionado, maximiza a separação entre classes, projetando os dados na direção que melhor discrimina diabéticos de não-diabéticos.

4.

LDA é claramente mais apropriado para construir uma regra discriminante, pois:

- LDA maximiza separabilidade entre classes (supervisionado), enquanto PCA apenas maximiza variância total (não-supervisionado);
- Separação visível: LDA mostra medianas bem distintas entre classes, o método PCA apresenta grande sobreposição;
- Objetivo alinhado: Para classificação, queremos discriminar classes, não apenas reduzir dimensionalidade.

Conclusão: LDA é o método adequado quando o objetivo é discriminação/classificação, reservando-se o PCA para exploração de dados ou de redução de dimensionalidade sem considerar classes.

 Generate

 Code

 Markdown